

Zynxis - Firebase Favorites

Database Schema & Persistence Deliverable

1. Feature Overview

The Zynxis Flutter application implements a Favorites feature using Firebase Cloud Firestore. A user can save a project by tapping the heart icon on the Zynxis Projects screen. The favorite is stored in Firestore and loaded again when the application is restarted.

2. Firestore Database Structure

Database: **(default)**

Collection: **favorites**

Document ID pattern: **project_{projectId}**

```
(default)
|
+-- favorites
  |
  +-- project_1
    | +-- projectId: 1
    | +-- title: String
    | +-- description: String
    | +-- imageUrl: String
    | +-- isFavorite: true
    | +-- savedAt: Timestamp
    |
  +-- project_2
    +-- projectId: 2
    +-- title: String
    +-- description: String
    +-- imageUrl: String
    +-- isFavorite: true
    +-- savedAt: Timestamp
```

3. Field Schema

Field	Type	Purpose
projectId	Number	Unique project identifier from the Projects API.
title	String	Stores the project's displayed title.
description	String	Stores the project's displayed description.
imageUrl	String	Stores the network image URL associated with the project.
isFavorite	Boolean	Records that the project is saved as a favorite.
savedAt	Timestamp	Records when the favorite was saved to Firestore.

4. Application Flow

Save: Zynxis Projects -> tap empty heart -> `FavoritesService.addFavorite()` -> Firestore creates/updates `favorites/project_ID` -> heart becomes filled.

Restore after restart: App starts -> Projects screen opens -> `FavoritesService.loadFavoriteIds()` reads the favorites collection -> saved project IDs are restored -> corresponding hearts are displayed as filled.

Remove: Tap a filled heart -> `FavoritesService.removeFavorite(projectId)` -> Firestore document is deleted -> heart returns to the empty state.

5. Persistence Verification

Verification procedure: save at least one project as a favorite, confirm its document appears under the Firestore favorites collection, completely stop the Flutter application, run the application again, return to Zynxis Projects, and confirm the same project remains marked with a filled heart. This demonstrates that the state is restored from Cloud Firestore rather than only from in-memory state.

6. Implementation Components

Component	Responsibility
<code>firebase_core</code>	Initializes the Firebase project when the Flutter app starts.
<code>cloud_firestore</code>	Provides Flutter access to Cloud Firestore.
<code>firebase_options.dart</code>	Contains platform-specific Firebase configuration generated by FlutterFire.
<code>favorites_service.dart</code>	Adds, removes, and loads favorite project records.
<code>api_projects.dart</code>	Displays projects and provides the heart control that calls <code>FavoritesService</code> .

Development note: Firestore was configured in test mode for the internship demo. Production deployment should use authenticated users and restrictive security rules.